

1. What is Postman and what is its primary purpose in API development?
2. How does Postman simplify the process of building and testing APIs?
3. What are the key functionalities provided by Postman for developers?
4. Which HTTP methods are supported by Postman?
5. How do these HTTP methods relate to CRUD operations in APIs?
6. What is the role of a GET request in API communication?
7. How does a POST request differ from a PUT request in Postman?
8. What types of data formats can be sent in Postman requests?
9. What is the difference between form-data, raw, and binary request formats?
10. What authentication methods are supported by Postman?
11. Why is authentication important when testing APIs?
12. What are collections in Postman and why are they useful?
13. How do collections improve collaboration in API development?
14. How can requests be organized within collections?
15. How does Postman support automated testing using scripts?
16. What programming language is used for writing test scripts in Postman?
17. How can Postman be integrated into CI/CD pipelines?
18. How can API documentation be generated using Postman?
19. What are the benefits of sharing workspaces in Postman?
20. How does Postman enable team collaboration during API development?
21. What is Node.js and why is it considered cross-platform?
22. Which JavaScript engine does Node.js use, and why is it important?
23. How does running JavaScript outside the browser benefit developers?

24. Why is Node.js considered highly performant compared to traditional server technologies?
25. How does Node.js handle multiple requests without creating new threads?
26. What is meant by a “single-process” model in Node.js?
27. How does Node.js avoid blocking operations during I/O tasks?
28. What role do asynchronous I/O primitives play in Node.js?
29. Why is blocking behavior considered rare in Node.js applications?
30. How does Node.js resume execution after completing an I/O operation?
31. How can Node.js handle thousands of concurrent connections efficiently?
32. What problem related to thread management does Node.js avoid?
33. Why is Node.js suitable for scalable network applications?
34. How does Node.js benefit frontend developers transitioning to backend development?
35. Why don't developers need to wait for browser updates to use new ECMAScript features in Node.js?
36. How can developers control the ECMAScript version used in Node.js?
37. What is the role of runtime flags in Node.js?
38. What types of I/O operations are commonly handled by Node.js?
39. How does Node.js improve CPU efficiency during waiting operations?
40. What is the basic concept behind a simple Node.js “Hello World” server example?
41. What is ReactJS and how does it differ from other frameworks like AngularJS?
42. What architectural pattern is commonly associated with ReactJS?
43. What is the primary role of components in ReactJS applications?
44. How is a React application rendered into the browser DOM?
45. What is the purpose of the `ReactDOM.render()` method?

46. What role does Babel play in the ReactJS toolchain?
47. How does Webpack contribute to ReactJS application development?
48. Why are React applications bundled into a single JavaScript file?
49. What must every React component's `render()` method return?
50. How does `React.createElement()` work in building UI elements?
51. What is JSX and how does it simplify React development?
52. How does JSX differ from plain HTML?
53. What is component state in React and how is it initialized?
54. How does `setState()` affect the rendering process?
55. Why must event handler functions sometimes be bound to the component instance?
56. Why does JSX require expressions instead of statements like `if` or `for`?
57. How can conditional rendering be implemented in JSX?
58. How is iteration over data handled in JSX?
59. What are lifecycle methods in React and why are they important?
60. What is the difference between class components and stateless functional components?